CHAPTER 6

Understanding Data Transfer Objects

Data Hiding, Response Aggregation, Client-Friendly Requests, and Mapping at the API Boundary in Spring Boot

Difficulty: Intermediate ⏱ Estimated Reading Time: 45 mins | #DTO #DataHiding #DataAggregation #RequestResponse #JavaRecords #SpringBoot

Chapter Overview

In the previous chapters, we built the backbone of the Library Management System: REST controllers that receive requests, Service classes that coordinate business operations, and repositories that persist domain objects. Throughout these chapters, our illustrative controllers returned and received **domain objects directly** in order to keep the examples focused on the layer being studied.

It is now time to examine that decision critically. Which objects should carry data between the client and the application? This chapter answers the question by introducing **Data Transfer Objects** (DTOs) through three practical benefits:

- **Data Hiding:** The client sees only the fields we deliberately choose to expose. Internal keys, sensitive data, and persistence details never leave the server.
- **Data Aggregation:** A single response can compose data from several entities, so the client receives a complete answer to its question in one request.
- **Controlled Input:** The client sends data through a flat, client-friendly contract that does not mirror the internal structure of our entities.

> **The Main Idea**
>
> A Data Transfer Object is a deliberate contract at the boundary of the application. The client sees what we choose to show, sends what we choose to accept, and never depends on how the domain model is implemented behind that boundary.

Learning Objectives

By the end of this chapter, you will be able to:

- ✓ Define a **Data Transfer Object** and explain its purpose.
- ✓ Explain the **data hiding** benefit: why internal keys and sensitive fields must not be returned to the client.
- ✓ Design response DTOs that expose only deliberately chosen fields.
- ✓ Explain the benefit of **flattening Value Objects** at the API boundary.
- ✓ Use response DTOs to **aggregate data** from several entities into one coherent answer.
- ✓ Explain why request DTOs should not mirror the structure of entities.
- ✓ Use different field names in the API contract and in the domain model.
- ✓ Recognize the **mass assignment** risk and describe how request DTOs prevent it.
- ✓ Place DTOs in the layered architecture and implement them with Java records.
- ✓ Map between DTOs and domain objects, and validate input at the boundary.

Prerequisites

Before studying this chapter, you should be familiar with:

- Java classes, constructors, records, and encapsulation.
- REST controllers and JSON request-response processing.
- The three-layer architecture and the responsibilities of controllers, services, and repositories (Chapter 5).
- Entities, Value Objects, and Aggregate Roots from Domain-Driven Design.
- Basic Bean Validation annotations such as `@NotBlank` and `@Email`.
- The Library Management System's Member, Loan, and Collection Bounded Contexts.

6.1 What Is a Data Transfer Object?

6.1.1 Definition

A **Data Transfer Object** (DTO) is a simple object whose purpose is to carry data between two parts of an application without carrying business behavior.

In a Spring Boot backend, DTOs most commonly cross the boundary between the client and the API:

- A **request DTO** carries data from the client into the application.
- A **response DTO** carries data from the application back to the client.

A DTO typically consists of fields and their accessors, and nothing else. It has no identity, no lifecycle, and no responsibility for enforcing rules.

6.1.2 What a DTO Is Not

It is equally important to understand what a DTO is not. A DTO is not an Entity, not a Value Object, and not a Service. Each of these concepts has a different purpose.

Table 6.1: DTOs compared with Entities and Value Objects

Characteristic	DTO	Entity	Value Object
Main Purpose	Carry data across a boundary.	Represent a domain concept with identity and lifecycle.	Represent a domain concept defined by its attributes.
Behavior	None, or only trivial helpers.	Domain behavior and controlled state transitions.	Domain behavior related to its own values.
Identity	Not meaningful.	Meaningful identity.	No identity; equality is defined by value.
Rules It Protects	None.	Domain invariants of the aggregate.	Consistency of its own values.

> **Important Distinction**
>
> An Entity answers the question "which business object is this?" A DTO answers the question "which data does this exchange contain?" The next three sections show why keeping these two concepts apart is essential.

6.2 The First Benefit: Data Hiding

6.2.1 Returning the Entity Directly

In the previous chapters, our illustrative controllers returned domain objects directly. Consider a controller that returns a `Member`:

```
@RestController
@RequestMapping("/api/v1/members")
@RequiredArgsConstructor
public class MemberController {

    private final MemberService memberService;

    @GetMapping("/{id}")
    public Member getMember(@PathVariable Long id) {
        return memberService.findMemberById(id);
    }
}
```

The `Member` entity can contain many fields such as the following:

```
@Entity
public class Member {

    @Id
    @GeneratedValue
    private Long id;                    // Internal database key

    private String memberNumber;        // Public library card number

    private String firstName;
    private String lastName;

    private String passwordHash;        // Sensitive credential data

    @Enumerated(EnumType.STRING)
    private MembershipTier tier;

    private boolean suspended;          // Internal membership state

    private int activeLoanCount;        // Internal derived counter

    @Version
    private long version;               // Persistence detail

    // Domain methods omitted.
}
```

But when the controller returns this object, Spring serializes **every field** into the response. The client receives:

```
{
  "id": 42,
  "memberNumber": "LIB-0042",
  "firstName": "Alice",
  "lastName": "Newman",
  "passwordHash": "$2a$10$X5wFBtLrL/kHcmrOGGTrGufsBX8CJ0Wp … ",
  "tier": "STANDARD",
  "suspended": false,
  "activeLoanCount": 2,
  "version": 7
}
```

6.2.2 Fields That Must Not Travel

This response contains fields that were never meant to be seen outside the server. They fall into three groups.

Internal identifiers. The `id` field is the database's primary key. It is an implementation detail of the persistence layer. Exposing it couples clients to our storage strategy and invites enumeration: a client who receives `"id": 42` can guess that `/api/v1/members/41` and `/api/v1/members/43` probably exist, and probe them. If clients need to reference a member later, the API should expose a **deliberate public identifier** such as the library card number `memberNumber`, not the raw database key.

Sensitive fields. The `passwordHash` must never leave the server. Even though it is hashed rather than stored in plain text, exposing it gives an attacker the material for offline password-cracking attempts. This field must be absent from every response, in every endpoint, forever.

Internal state and persistence details. Fields such as `suspended`, `activeLoanCount`, and `version` describe internal bookkeeping. They are not part of what a client needs in order to use the library, and exposing them invites clients to depend on internals that may change.

Table 6.2: Deciding field by field what the API may expose

Field	Expose?	Reason
`id`	No	Internal surrogate key; invites enumeration and couples clients to storage.
`memberNumber`	Yes	Stable public identifier (the library card number).
`firstName`, `lastName`	Yes	Display data the client needs.
`email`	Yes	Contact data relevant to the client.
`passwordHash`	Never	Secret credential material; must not leave the server.
`tier`	Yes	Meaningful membership information.
`suspended`	No	Internal state; expose through a dedicated use case if ever needed.
`activeLoanCount`	No	Internal derived counter.
`version`	No	Optimistic-locking detail of the persistence layer.

6.2.3 The Response DTO: Exposure Becomes a Decision

The remedy is to return a different object: one that contains only the required fields and omits every field we need to keep hidden.

```
public record MemberResponse(
        String memberNumber,
        String firstName,
        String lastName,
        String email,
        MembershipTier tier
) {}
```

We declare DTOs as Java **records**, a concise way to define immutable data carriers. Section 6 explains this choice in full. The controller now returns the DTO:

```
@GetMapping("/{memberNumber}")
public MemberResponse getMember(@PathVariable String memberNumber) {
    Member member = memberService.findMemberByNumber(memberNumber);
    return toResponse(member);   // Mapping is explained in Section 7.
}
```

And the client receives:

```
{
  "memberNumber": "LIB-0042",
  "firstName": "Alice",
  "lastName": "Newman",
  "email": "alice@example.org",
  "tier": "STANDARD"
}
```

The response is now smaller, safer, and more meaningful. Note also that the URL addresses the member by its public identifier `memberNumber` rather than by the hidden database key.



FIGURE 6.1: Returning the entity exposes everything; the response DTO exposes only what was chosen.

> **Exposure Becomes a Decision**
>
> With entities, exposure is an accident of the persistence model: whatever exists in the class ends up in the JSON. With a DTO, exposure is a decision recorded in code. A reviewer can open `MemberResponse` and see, at a glance, exactly what leaves the system , and confirm that `passwordHash` is not among it.

6.2.4 Flattening Value Objects

Our domain model becomes richer over time. Fields that matter to the business are often modeled as **Value Objects** rather than plain strings:

```
public record EmailAddress(String value) {}

public record Address(String street, String city) {}

public class Member {
    // ...
    private EmailAddress email;    // Value Object
    private Address address;       // Value Object
}
```

If the entity were serialized directly, the client would receive the internal shape of these Value Objects:

```
{
  "email": { "value": "alice@example.org" },
  "address": {
    "street": "12 Oak Avenue",
    "city": "Springfield"
  }
}
```

This is problematic for several reasons:

- The client must navigate wrappers such as `email.value`, which are meaningless outside the domain model.
- The internal structure of the Value Objects becomes part of the API contract. Renaming `EmailAddress.value` tomorrow would break every client.
- The nesting adds complexity to every client, including mobile applications and third-party integrations.

The response DTO **flattens** the Value Objects into simple fields:

```
public record MemberResponse(
        String memberNumber,
        String firstName,
        String lastName,
        String email,          // flattened from EmailAddress
        String city,           // flattened from Address
        MembershipTier tier
) {}
```

```
{
  "memberNumber": "LIB-0042",
  "firstName": "Alice",
  "lastName": "Newman",
  "email": "alice@example.org",
  "city": "Springfield",
  "tier": "STANDARD"
}
```

Flattening and selection work together. In this example, the `Address` Value Object is not only flattened but also **partially exposed**: the client sees the `city` and not the full address, because this use case does not require it. The domain keeps its rich model; the contract stays simple.

> **The Domain Stays Rich, the Contract Stays Simple**
>
> Flattening is performed by the mapping code at the boundary, not by annotations on the Value Objects. The domain model never needs to know how it will be presented to clients.

> **A Stable Contract**
>
> Because clients depend on `MemberResponse` and not on `Member`, the domain model can evolve freely: fields can be renamed, Value Objects can be restructured, and new columns can be added , while the JSON the client receives remains unchanged.

6.3 The Second Benefit: Data Aggregation

6.3.1 One Entity Is Not Always Enough

Data hiding solves the problem of returning **too much**. But sometimes a single entity returns **too little**. Consider a client asking the Library Management System: *"show me this loan."* The `Loan` entity stores references to the borrower and the item by identity, not by carrying their data:

```
public class Loan {

    private LoanId id;

    private MemberId borrowerId;    // Reference by identity
    private ItemId itemId;          // Reference by identity

    private String loanNumber;      // Public identifier
    private LocalDate borrowingDate;
    private LocalDate dueDate;
    private LoanStatus status;
}
```

If we returned this entity directly, the client would receive:

```
{
  "loanNumber": "LN-000731",
  "borrowerId": 42,
  "itemId": 17,
  "borrowingDate": "2026-10-01",
  "dueDate": "2026-10-15",
  "status": "ACTIVE"
}
```

The numbers `42` and `17` are meaningless to the client. Who borrowed the item? Which book is it? To answer the client's actual question, the response must combine data from **three** domain objects: the Loan, the Member, and the LibraryItem.

One might be tempted to solve this by letting `Loan` hold direct object references to `Member` and `LibraryItem` and serializing the whole graph. That approach reintroduces every problem from the previous section , the full `Member`, including its hidden fields, would pour into the response, and adds new ones, such as infinite recursion between bidirectional references and lazy-loading failures during serialization.

6.3.2 The Response DTO as a Composition

A response DTO can **aggregate** data from several entities into one coherent answer:

```
public record LoanResponse(
        String loanNumber,
        String memberName,       // from Member
        String bookTitle,        // from LibraryItem
        String isbn,             // from the Book details
        LocalDate borrowingDate, // from Loan
        LocalDate dueDate,       // from Loan
        LoanStatus status        // from Loan
) {}
```

One response object, three sources. Notice that the fields are also **chosen** and **flattened**: the client sees the member's name but not the member's internal state, and the book's title but not the item's persistence details.

6.3.3 Assembling the Response using a Mapper class

All of this is great, but recall that the controller delegates all reauests to the servce layer. The servive layer returns domain objects and not DTOs. So the controller needs to convert the domain objects into a DTO. In other words we need to write a lot of code in the controller to convert domain objects into DTOs. This will make the controller classes bloated with code and hard to read. Also, a lot of this code is repetitive and can be reused in other controllers. Therefore a better approach is to create another class whose sole purpose is to convert domain objects into DTOs. We call this class a **Mapper** class. The Mapper class will have methods that take domain objects as input and return DTOs as output. For example, the following Mapper class has a method that takes a Loan, Member and LibraryItem as input and returns a LoanResponse DTO as output.

```
@Component
public class LoanMapper {

    public LoanResponse toResponse(
            Loan loan, Member member, LibraryItem item) {

        return new LoanResponse(
                loan.getLoanNumber(),
                member.getFirstName() + " " + member.getLastName(),
                item.getTitle(),
                item.getIsbn(),
                loan.getBorrowingDate(),
                loan.getDueDate(),
                loan.getStatus()
        );
    }
}
```

Retrieving the three objects is part of the use case, so the Service coordinates it:

```
@Service
@RequiredArgsConstructor
public class LoanService {

    private final LoanRepository loanRepository;
    private final MemberRepository memberRepository;
    private final LibraryItemRepository itemRepository;
    private final LoanMapper loanMapper;

    public LoanResponse findLoanDetails(LoanId loanId) {

        Loan loan = loanRepository.findById(loanId)
                .orElseThrow(() → new LoanNotFoundException(loanId));

        Member member = memberRepository.findById(loan.getBorrowerId())
                .orElseThrow(() →
                    new MemberNotFoundException(loan.getBorrowerId()));

        LibraryItem item = itemRepository.findById(loan.getItemId())
                .orElseThrow(() →
                    new ItemNotFoundException(loan.getItemId()));

        return loanMapper.toResponse(loan, member, item);
    }
}
```

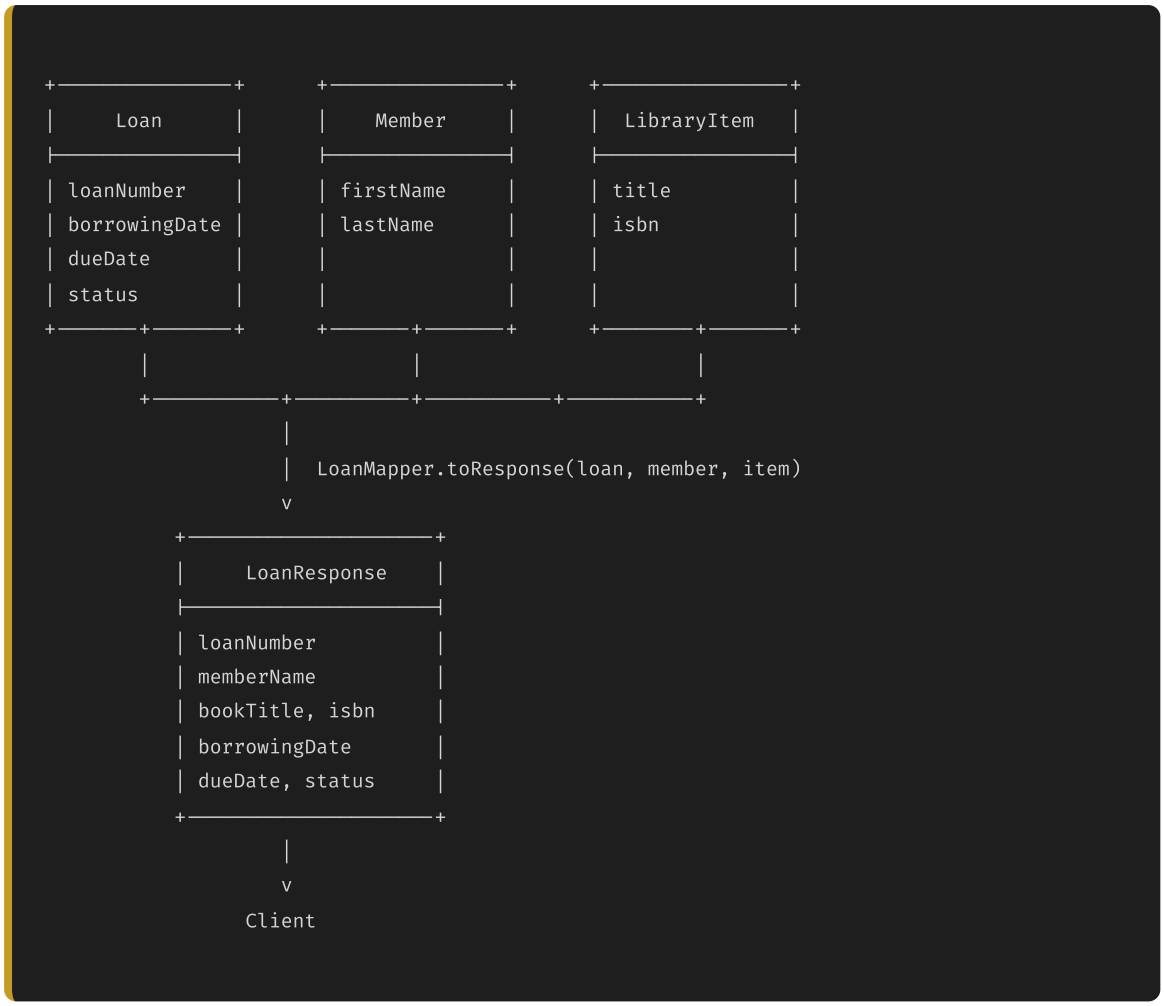


FIGURE 6.2: One response DTO aggregating data from three domain objects.

6.3.4 Aggregation Is a Read-Side Concern

Notice what did **not** happen: the `Loan` aggregate was not modified, no bidirectional references were added, and no object graph was serialized. The aggregation produced a **view** , a read-only composition built to answer one question well.

> **A View, Not a Domain Object**
>
> Adding a field to `LoanResponse` does not touch the `Loan` aggregate. The response DTO can be reshaped per use case , a list view, a detail view, a receipt , while the domain model remains stable.

> **A Note on Bounded Contexts**
>
> Each Bounded Context can publish the summaries it needs. The Loan Management context may define a `MemberSummary` containing only the name and borrowing eligibility, while the Member Management context owns the full `MemberResponse`. Context-specific DTOs keep the contexts independent, just as context-specific domain models do.

6.4 The Third Benefit: Controlled Input with Request DTOs

The same principles apply in the opposite direction. Just as the client should not receive our entities, the client should not have to **send** our entities either.

6.4.1 The Mirror Problem

Suppose registering a member required the client to submit the `Member` entity's own structure. The request body would have to look like this:

```
{
  "id": null,
  "memberNumber": null,
  "firstName": "Alice",
  "lastName": "Newman",
  "email": { "value": "alice@example.org" },
  "address": { "street": "12 Oak Avenue", "city": "Springfield" },
  "tier": "STANDARD",
  "suspended": false,
  "activeLoanCount": 0,
  "version": 0
}
```

This request is problematic:

- The client must know the **internal structure of our Value Objects**, including wrappers such as `email.value`.
- The client must supply **internal fields it should never control**: `id`, `version`, `activeLoanCount`, and even `suspended`.
- The contract is **fragile**: restructuring a Value Object in the domain breaks every client.
- There is no room for API-friendly naming, since the field names are dictated by the entity.

6.4.2 A Flat, Client-Friendly Request

A request DTO inverts the mirror. The client sends a flat, simple structure, and the mapping code reassembles the domain's Value Objects on the server side:

```
public record CreateMemberRequest(
        @NotBlank String firstName,
        @NotBlank String lastName,
        @Email @NotBlank String email,
        @NotBlank String street,
        @NotBlank String city,
        @NotNull MembershipTier tier
) {}
```

```
{
  "firstName": "Alice",
  "lastName": "Newman",
  "email": "alice@example.org",
  "street": "12 Oak Avenue",
  "city": "Springfield",
  "tier": "STANDARD"
}
```

Compare this with the mirror-shaped request above. There is no `id`, no `version`, no `suspended`, no `activeLoanCount`. The server assigns all internal state; the client never does. The request is also symmetric with the response: the client sends flat data and receives flat data, while the domain keeps its rich structure on both sides.

6.4.3 Different Names for the Same Data

Because the request DTO is a separate contract, its field names do not have to match the domain's field names. The API vocabulary can be chosen for clarity, and the mapping translates between the two vocabularies.

```
public record RegisterMemberRequest(
        @NotBlank String firstName,
        @NotBlank String lastName,
        @Email  @NotBlank String emailAddress,
        @NotBlank String password,
        @NotNull MembershipTier membershipTier
) {}
```

Table 6.3: The API vocabulary and the domain vocabulary may differ deliberately

Client Sends	Domain Receives	Why the Names Differ
`emailAddress`	`EmailAddress email`	The API favors explicit, self-descriptive names; the domain favors its own established terms.
`membershipTier`	`tier`	Each side may use the name that reads best in its own context.
`password`	`passwordHash` (after hashing)	A security transformation happens in the Service. The DTO carries the password only for the duration of the request; the entity stores only the hash; no response ever contains either.

The third row deserves emphasis. The request DTO is what makes this transformation clean: the client sends `password`, the Service hashes it as part of the use case, and the entity stores `passwordHash`. The plain-text password exists only inside the request, and the hash never travels back out , the response DTO defined in Section 2 has no field for it.

6.4.4 Protection Against Mass Assignment

There is also a security dimension to controlled input. If a controller binds the incoming JSON directly onto the entity, then **every entity property becomes client-settable**. A malicious or careless client could submit:

```
{
  "firstName": "Alice",
  "lastName": "Newman",
  "email": "alice@example.org",
  "tier": "PREMIUM",
  "suspended": false,
  "activeLoanCount": 999
}
```

Bound to the entity, the fields `tier`, `suspended`, and `activeLoanCount` would be silently applied , a client upgrading its own membership and resetting its suspension status. Bound to `CreateMemberRequest`, those fields simply do not exist in the contract. **The payload cannot set what the object does not contain.**

> **Never Bind a Request Body to an Entity**
>
> Binding `@RequestBody` directly to an entity hands the client the keys to internal state. Always receive a request DTO, and let the Service decide which internal fields are initialized and how.

6.5 Where Do DTOs Fit in the Layered Architecture?

6.5.1 Boundary Objects

DTOs are **boundary objects**. They live at the edge of the application, alongside the presentation layer. The controller receives a request DTO, hands domain input to the service, and converts the domain result into a response DTO. The service and the repositories never see the request or response DTOs.

6.5.2 Architectural Diagram





FIGURE 6.3: DTOs belong to the boundary; domain objects stay inside.

6.5.3 Who Creates and Consumes DTOs?

TABLE 6.4: Production and consumption of the boundary contracts

DTO	CREATED BY	CONSUMED BY
`CreateMemberRequest`	Spring, by deserializing the request body.	The controller, which maps it to domain input.
`BorrowItemRequest`	Spring, by deserializing the request body.	The controller, which passes identifiers to the service.
`MemberResponse`	The mapper, from a `Member` domain object.	Spring, which serializes it to JSON for the client.
`LoanResponse`	The mapper, from a `Loan`, a `Member`, and a `LibraryItem`.	Spring, which serializes it to JSON for the client.

> **THE CONTRACT OF THE BOUNDARY**
>
> The DTOs are the application's published language at its edge. They tell clients what they may send and what they will receive, independently of how the domain model is implemented behind the boundary.

6.6 How Do We Implement DTOs in Java?

6.6.1 DTOs as Records

Since Java 16, **records** provide a concise way to define immutable data carriers. A record declares its components in the header and generates the constructor, accessors, `equals`, `hashCode`, and `toString` automatically.

```
public record MemberResponse(
        String memberNumber,
        String firstName,
        String lastName,
        String email,
        MembershipTier tier
) {}
```

Validation annotations can be placed directly on record components:

```
public record CreateMemberRequest(
        @NotBlank String firstName,
        @NotBlank String lastName,
        @Email @NotBlank String email,
        @NotNull MembershipTier tier
) {}
```

6.6.2 DTOs as Classes

A DTO can also be a regular Java class with private fields and accessors. This approach works with every Java version and every serialization library.

```
public class MemberResponse {

    private final String memberNumber;
    private final String firstName;
    private final String lastName;
    private final String email;
    private final MembershipTier tier;

    public MemberResponse(
            String memberNumber,
            String firstName,
            String lastName,
            String email,
            MembershipTier tier) {

        this.memberNumber = memberNumber;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.tier = tier;
    }

    // Getters omitted.
}
```

6.6.3 Why Immutability Fits DTOs

- **Safety:** A DTO should not change after it has been created. Records enforce this by design.
- **Concision:** The entire definition fits in a few lines, keeping boundary classes easy to review.
- **Value semantics:** Records implement equality by component values, which is convenient in tests.
- **Framework support:** Modern versions of Spring Boot and Jackson support records for both deserialization and serialization.

> **When to Use a Class Instead**
>
> A regular class may be preferable when a framework or tooling requires a no-argument constructor with settable fields, when the DTO benefits from a builder, or when the project targets an older Java version. Records are the default recommendation for new code.

6.6.4 Types Inside DTOs

DTO fields are commonly limited to simple, serializable types such as `Long`, `String`, enums, and date types. These types map naturally to JSON.

The domain may model identifiers as Value Objects such as `MemberId` , as the Service signatures in Chapter 5 did. At the boundary, DTOs carry the simple representation, and the mapping code wraps or unwraps the Value Object as needed:

```
// Mapping between the two representations.
MemberId memberId = new MemberId(request.memberId());
Long rawId = member.getId().value();
```

Other designs pass domain value objects through the boundary directly. The important principle is that the decision is explicit and applied consistently across the API.

6.7 How Do We Map Between DTOs and Domain Objects?

6.7.1 What Is Mapping?

Mapping is the translation of data between the two representations we have been discussing: from a request DTO to domain input, and from domain objects to a response DTO. Mapping unwraps Value Objects on the way in, and flattens and aggregates them on the way out.

Mapping is a mechanical activity. It copies and converts values; it does not decide whether an operation is allowed. Mapping is not business logic.

6.7.2 Where Should Mapping Happen?

Table 6.5: Two approaches to placing the mapping responsibility

Approach	Who Maps	Trade-offs
Controller-side mapping	The controller maps request DTOs to domain input and domain results to response DTOs.	The service remains independent of the API contract and reusable by other entry points. Slightly more code in the controllers.
Service-side mapping	The service accepts and returns DTOs directly.	Fewer moving parts, but the service becomes coupled to the API contract, and non-HTTP entry points must use the same DTOs.

Consistent with the separation of responsibilities discussed in Chapter 5, this book prefers controller-side mapping: the service speaks the language of the domain, while the controller translates at the boundary. What matters most is that a team applies one approach consistently.

6.7.3 A Mapper Class

Mapping code should be centralized rather than scattered across controllers. A dedicated mapper class keeps the translation , including the flattening and reassembly of Value Objects , in one place:

```
@Component
public class MemberMapper {

    // Incoming: reassemble Value Objects from flat request data.
    public Member toDomain(CreateMemberRequest request) {
        return new Member(
                new PersonName(request.firstName(), request.lastName()),
                new EmailAddress(request.email()),
                new Address(request.street(), request.city()),
                request.tier()
        );
    }

    // Outgoing: flatten and select.
    public MemberResponse toResponse(Member member) {
        return new MemberResponse(
                member.getMemberNumber(),
                member.getFirstName(),
                member.getLastName(),
                member.getEmail().value(),   // unwrap the Value Object
                member.getAddress().city(),  // expose only part of it
                member.getTier()
        );
    }
}
```

> **A Note on Domain Construction**
>
> Some designs prefer that the mapper produce a dedicated input object, and that a domain factory or the Service construct the Entity. This keeps construction rules and invariant checks inside the domain, consistent with the aggregate protection discussed in Chapter 5. The simplified example above is acceptable for illustrations, but the domain should always have the final word on what constitutes a valid object.

6.7.4 The Controller, Refined

With a mapper in place, the controller from the beginning of this chapter can be refined:

```
@RestController
@RequestMapping("/api/v1/members")
@RequiredArgsConstructor
public class MemberController {

    private final MemberService memberService;
    private final MemberMapper memberMapper;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public MemberResponse registerMember(
            @Valid @RequestBody CreateMemberRequest request) {

        Member member = memberService.registerMember(
                memberMapper.toDomain(request));

        return memberMapper.toResponse(member);
    }

    @GetMapping("/{memberNumber}")
    public MemberResponse getMember(@PathVariable String memberNumber) {

        Member member = memberService.findMemberByNumber(memberNumber);

        return memberMapper.toResponse(member);
    }
}
```

The controller now speaks two languages: DTOs toward the client, and domain objects toward the service. Neither side sees the other's representation.

6.7.5 The Complete Flow

```
Client
  |
  | POST /api/v1/members
  | { "firstName": "Alice", "lastName": "Newman",
  |   "email": "alice@example.org", "tier": "STANDARD" }
  v
CreateMemberRequest
  |
  | Deserialized and validated at the boundary
  v
MemberController
  |
  | memberMapper.toDomain(request)
  | Value Objects reassembled
  v
MemberService
  |
  | Apply business rules
  | Create the Member
  v
MemberRepository
  |
  | Persist changes
  v
Member
  |
  | memberMapper.toResponse(member)
  | Value Objects flattened, hidden fields omitted
  v
MemberResponse
  |
  | Serialized to JSON
  v
Client
```

Figure 6.4: The flow of data through request and response DTOs.

> **A Note on Mapping Libraries**
>
> Libraries such as MapStruct can generate mapper implementations from declarative interface definitions, reducing boilerplate for large projects. Manual mapping remains a perfectly valid and transparent choice, especially while learning.

> **Mapping Is Not Business Logic**
>
> If mapping code starts making decisions , choosing statuses, computing eligibility, or applying rules , that logic belongs in a Service or a domain object, not in the mapper. Mapping translates; it does not decide.

6.8 Validating Request DTOs at the Boundary

6.8.1 Declaring Input Constraints

Request DTOs are the natural place to declare the constraints that incoming data must satisfy. Bean Validation annotations express these constraints directly on the DTO fields, and the controller activates them with the `@Valid` annotation:

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public MemberResponse registerMember(
        @Valid @RequestBody CreateMemberRequest request) {
    // Spring only invokes this method when the
    // request DTO satisfies its constraints.
}
```

When validation fails, Spring rejects the request before the controller method is invoked, and a centralized exception handler can translate the validation failure into an appropriate HTTP 400 Bad Request response:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<String> handleValidationFailure(
        MethodArgumentNotValidException ex) {

    return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body("The submitted request is invalid.");
}
```

6.8.2 Validation Answers a Familiar Question

In Chapter 5, we asked which mechanism should detect a borrowing request with a missing member identifier. With request DTOs, the answer becomes concrete:

```
public record BorrowItemRequest(
        @NotNull Long memberId,
        @NotNull Long itemId
) {}
```

The `@NotNull` constraint on the request DTO rejects the incomplete request at the boundary, before any business operation is attempted. The service never has to defend itself against structurally invalid input.

6.8.3 Validation Is Not Invariant Enforcement

Table 6.6: The three mechanisms revisited from Chapter 5

Mechanism	Main Question	Where It Lives
Validation	Is the input acceptable?	At the boundary, on request DTOs.
Invariant Enforcement	Does the domain remain in a valid state?	In domain objects, aggregates, and services.
Exception Handling	How are exceptional conditions communicated?	In handlers at appropriate boundaries.

A request can pass every validation constraint and still be rejected by the business logic. A borrowing request may contain a well-formed member identifier and a well-formed item identifier, while the member is suspended and therefore not eligible to borrow. Eligibility is a business rule, not an input constraint.

> **Important**
>
> Validation on a request DTO checks the shape and format of incoming data. It does not, by itself, protect the consistency of the domain model. Both mechanisms are necessary, and they operate at different boundaries.

6.9 Common Mistakes When Working with DTOs

Recognizing anti-patterns is as valuable as knowing the correct approach. The following mistakes appear frequently in backend projects.

Table 6.7: Frequent DTO anti-patterns and their remedies

Mistake	Why It Is a Problem	Better Approach
Returning entities from controllers.	Exposes internal keys and sensitive fields, couples the API to persistence, and leaks data by accident.	Map domain results to response DTOs with deliberately chosen fields.
Binding request bodies directly to entities.	Allows mass assignment of internal state and forces clients to send the entity's structure.	Receive a flat request DTO per use case.
Mirroring Value Object structure in the API.	Exposes wrappers such as `email.value` and makes domain refactoring a breaking change.	Flatten Value Objects into simple fields at the boundary.
Letting responses answer with raw identifiers only.	Clients receive meaningless keys and must issue more requests to understand the data.	Aggregate the data the client actually needs into the response.
Business logic inside DTOs.	Scatters rules into objects that have no domain responsibility.	Keep rules in Services and domain objects; keep DTOs passive.
Entity constructors that accept request DTOs.	Makes the domain depend on the API layer, inverting the dependency direction.	Map in boundary-side mapper classes; let the domain control construction.
Adding Jackson annotations to entities to fix serialization.	Solves the symptom while coupling the domain to the presentation framework.	Return DTOs shaped for the client, and leave entities unaware of the API.

6.10 Best Practices

Pro Tip 1: Treat Every Response Field as a Decision

A response DTO should read like a decision record. Opening `MemberResponse` should reveal exactly what leaves the system , and make it obvious that internal keys and sensitive fields are not among them.

Pro Tip 2: Never Return Entities, Even in Prototypes

The shortcut of returning an entity creates coupling and exposure that are difficult to undo later. The cost of a small response DTO is always lower than the cost of a leaked `passwordHash`.

Pro Tip 3: Flatten Value Objects at the Boundary

Keep the domain model rich, and translate to simple, flat fields at the boundary. The client should never need to navigate wrappers such as `email.value`.

Pro Tip 4: Compose Views, Do Not Navigate Associations

When a response needs data from several entities, aggregate it explicitly in the Service and mapper. Do not serialize object graphs, and do not add bidirectional references to the domain model to satisfy a view.

Pro Tip 5: Keep Request DTOs Flat and Client-Friendly

The client should send simple fields with clear names, never the entity's internal structure. Internal fields such as identifiers, counters, and state flags are assigned by the server, not by the client.

Pro Tip 6: Validate at the Boundary, Enforce Invariants in the Domain

Use Bean Validation on request DTOs to check input constraints, and rely on domain objects and services to enforce invariants. The two mechanisms complement each other and must not be confused.

Review Questions

Q1. What is a Data Transfer Object? What problem does the pattern solve?

Q2. A controller returns a `Member` entity directly. List the fields that reach the client, and classify each as acceptable, internal, or sensitive.

Q3. Why should the internal primary key `id` not be exposed? What should be exposed instead when clients need to reference a member later?

Q4. The `passwordHash` is already hashed rather than stored in plain text. Why must it still never appear in a response?

Q5. What does it mean to say that "exposure becomes a decision" when a response DTO is used?

Q6. What is flattening? Using the `Address` Value Object as an example, show the nested JSON a client would receive without flattening, and the flat JSON a client receives with it.

Q7. Why should Value Objects remain rich inside the domain while being flattened at the boundary? Who performs the flattening?

Q8. What is data aggregation? In the `LoanResponse` example, state which fields come from the `Loan`, which from the `Member`, and which from the `LibraryItem`.

Q9. Why is returning a `Loan` with only `borrowerId` and `itemId` insufficient for the client? Why is serializing the object graph not a good alternative?

Q10. Explain why aggregation is a read-side concern. What happens , and what does not happen , to the `Loan` aggregate when a new field is added to `LoanResponse`?

Q11. Why should a request DTO not mirror the structure of the entity? Give two reasons, one related to Value Objects and one related to internal fields.

Q12. What is mass assignment? How does a request DTO prevent a client from setting `suspended` or `tier`?

Q13. Give two reasons why an API field name might differ from the corresponding domain field name. Use `emailAddress` and `password` in your answer.

Q14. In the registration flow, where does the transformation from `password` to `passwordHash` happen, and why is that location appropriate?

Q15. What are Java records? Why are they well suited for implementing DTOs, and when might a regular class be preferable?

Q16. Compare controller-side mapping and service-side mapping. What are the trade-offs of each approach?

Q17. A client submits a borrowing request without a member identifier. Which mechanism detects this problem, and where does the detection happen? (Connect your answer to Chapter 5.)

Q18. A borrowing request contains a well-formed member identifier, but the member is suspended. Why is this not a validation problem, and which mechanism rejects the operation?

Chapter Summary

In this chapter, we introduced Data Transfer Objects and examined the three benefits they bring to the boundary of a Spring Boot application.

Data hiding ensures that the client sees only deliberately chosen fields. Internal primary keys are replaced by public identifiers such as `memberNumber`, sensitive fields such as `passwordHash` never leave the server, and Value Objects are flattened into simple, client-friendly fields.

Data aggregation allows one response to compose data from several entities. A `LoanResponse` combines information from the Loan, the Member, and the LibraryItem, giving the client a complete answer to its question without exposing object graphs or raw identifiers.

Controlled input applies the same principles in the opposite direction. Request DTOs are flat, carry client-friendly names, omit internal fields entirely, and protect the application from mass assignment.

We also placed DTOs in the layered architecture, implemented them with Java records, centralized the translation in mapper classes, and distinguished validation at the boundary from invariant enforcement in the domain.

The Big Picture



The client speaks flat JSON in both directions. The boundary speaks DTOs. The domain speaks the language of the business. Mapping connects these languages without allowing one to leak into another.

Up Next

In this chapter, we studied the concepts behind Data Transfer Objects: data hiding, aggregation, and controlled input.

In the next practical activity, we will apply these concepts to the Library Management System by defining request and response DTOs for the Member Management, Loan Management, and Collection Management contexts, implementing mapper classes, and connecting the controllers to the Service layer through these contracts.

We will combine the Service classes from the previous chapter with the DTOs from this chapter, and observe how validation at the boundary and invariant enforcement in the domain work together during the execution of a complete business operation.